**Patent Document 2**

# MEGA — The Maximizing Expected Generalization Algorithm for Learning Complex Query Concepts (Morpho Software Confidential)

**Edward Chang**
Morpho Software Inc.
3944 State Street, #340
Santa Barbara, CA 93105
echang@db.stanford.edu

March 21, 2001

**Abstract**

Specifying exact query concepts has become increasingly challenging to end-users. This is because many query concepts (e.g., those for looking up a multimedia object) can be hard to articulate, and articulation can be subjective. In this study, we propose a query-concept learner that learns query criteria through an intelligent sampling process. Our concept learner aims to fulfill two primary design objectives: 1) it has to be expressive in order to model most practical query concepts, and 2) it must learn a concept quickly and with a small number of labeled data since on-line users tend to be too impatient to provide much feedback. To fulfill the first goal, we model query concepts in $k$-CNF, which can express almost all practical query concepts. To fulfill the second design goal, we propose our **m**aximizing **e**xpected **g**eneralization **a**lgorithm (MEGA), which converges to target concepts quickly by its two complementary steps: sample selection and feature reduction. MEGA copes well with moderate noise and it uses a multi-resolution/hierarchical learning method to effectively deal with very large database issues. Through examples, analysis and experiments, we show that MEGA converges to query concepts significantly faster than traditional methods.

**Keywords**: query concept, relevance feedback, active learning.

## 1 Introduction

The query-concept learning approach that we research in this study can be characterized by the following example: Suppose one is asked, "Are the paintings of Leonardo da Vinci more like those of Peter Paul Rubens or those of Raphael?" One is likely to respond with: "What is the basis for the comparison?" Indeed, without knowing the criteria (i.e., the query concept) by which the comparison is to be made, a database system cannot conduct a search. For many search tasks, however, a query

concept is hard to articulate, and articulation can be subjective. For instance, in a multimedia search, it is difficult to describe a desired image using low-level features such as color, shape and texture (these are widely used features for representing images [17]). Different users may use different combinations of these features to depict the same image. In addition, most users (e.g., Internet users) are not trained to specify simple query criteria using SQL, for instance. In order to take individuals' subjectivity into consideration and to make information access easier, it is both necessary and desirable to build intelligent search engines that can discover (i.e., that can learn) individuals' query concepts quickly and accurately.

To learn users' query concepts, we propose a query-concept learner that learns a concept through an intelligent sampling process. The design of a query-concept learner must fulfill two primary goals. One, the concept-learner's hypothesis space must not be too restrictive, so it can model most practical query concepts. Two, the concept-learner should grasp a concept quickly and with a small number of labeled instances, since most users do not wait around to provide a great deal of feedback. To fulfill these design goals, we propose the *Maximizing Expected Generalization Algorithm* (MEGA). MEGA models query concepts in $k$-CNF [8], which can model almost all practical query concepts[1]. To ensure that target concepts can be learned quickly and with a small number of samples, MEGA employs two methods: sample selection (S-step) and feature reduction (F-step). In its S-step, MEGA judiciously selects samples that can collect maximum information from users to remove irrelevant features in its subsequent F-step. In its F-step, MEGA removes irrelevant terms from the query-concept (i.e., a $k$-CNF), and at the same time, refines the sampling boundary (i.e., a $k$-DNF) so that most informative samples can be selected in its subsequent S-step. Unlike traditional query refinement methods, which uses only the S-step or only the F-step (Section 5 highlights related work), MEGA uses these two steps in a complementary way to achieve fast convergence to target concepts.

To learn a $k$-CNF efficiently, MEGA uses a multi-resolution/hierarchical learning method. MEGA first divides features into subgroups of different resolutions. It then explores the structure of the resolution hierarchy to reduce learning space and time complexity. We show that when features are divided carefully into $G$ groups, MEGA can achieve a speedup of $O(G^{k-1})$ with little precision loss.

We compare MEGA with five sampling schemes: *random*, *bounded random*, *nearest neighbor*, *query expansion* and *aggressive*. (These sampling schemes reflect existing work in query refinement.) Our evaluation shows that MEGA learns query concepts substantially faster than these traditional schemes. Our experiments with a high-dimensional image dataset confirms the effectiveness of MEGA in learning complex query concepts both accurately and quickly. We believe that a scheme like MEGA, which can model most query concepts and can learn them quickly can make information access easier, faster, and more personal and user-friendly.

[1] $k$-CNF is more expressive than $k$-term DNF, and it has both polynomial sample complexity and time complexity [9, 13].

### 1.1 Motivating Example

Below we show an example problem that illustrates the need for MEGA. (We will use this simple example throughout this paper to explain various aspects of our approach and to contrast our approach with others. In Section 4 we show that MEGA works well to learn complex query concepts for a high-dimensional image dataset.)

Suppose Jane plans to apply to a graduate school. Before filling out the forms and paying the application fees, she would like to estimate her chances of being admitted. Since she does not know the admission criteria, she decides to learn the admission concept by induction[2]. She calls up a few friends who applied last year and obtains the information shown in Table 1.

| *Name* | *GPA* | *GRE* | *Has Publications?* | *Is Athletic?* | *Was Admitted?* |
|---|---|---|---|---|---|
| *Joe* | *high* | *high* | *false* | *true* | *true* |
| *Mary* | *high* | *low* | *true* | *false* | *true* |
| *Emily* | *high* | *low* | *true* | *true* | *true* |
| *Lulu* | *high* | *high* | *true* | *true* | *true* |
| *Anna* | *low* | *low* | *true* | *false* | *false* |
| *Peter* | *low* | *high* | *false* | *false* | *false* |
| *Mike* | *high* | *low* | *false* | *false* | *false* |
| *Pica* | *low* | *low* | *false* | *false* | *false* |

Table 1: Admission Samples.

If we look at the GRE scores in the table, we see that students with either high or low GRE scores were admitted, also both kinds were rejected. Hence, we may conclude that the GRE is irrelevant in the admission process. Likewise, one's publication record does not affect admission acceptance, nor does having a high GPA. It may appear that the admission decision is entirely random. However, the graduate school actually uses a combination of reasonable criteria: it requires a high GPA and either a high GRE or publications. In other words, $Admission : GPA = high \wedge (GRE = high \vee Publications = true)$.

Two obvious questions arise: "Are all the samples in Table 1 equally useful for learning the target concept?" and, "Are all features in the table relevant to the learning task?"

- Are all samples equally useful? Apparently not, for several reasons. First, it seems that *Pica*'s record may not be useful since she was unlikely to be admitted (i.e., her record is unlikely to be labeled positive). Second, both Emily and Mary have the same record, so one of these two records is redundant. Third, Lulu's record is perfect and hence does not provide additional insight for learning the admission criteria. This example indicates that choosing samples randomly may not produce useful information for learning a target concept.

[2]At first sight, this example may seem very different from, say, an image search scenario, where a user queries similar images by example(s). But if we treat the admission officer as the user who knows what he/she likes and who can, accordingly, label a data as *true* or *false*, and if we treat Jane as the search engine who tries to find out what the admission officer thinks, then it is evident that this example represents a typical search scenario.

- Are all features relevant? To determine relevancy, we examine the features in the table. The feature *"Is athletic?"* does not seem to be relevant to graduate admissions. The presence of irrelevant features can slow down concept learning exponentially [10, 11].

In this paper, we will show how and why MEGA can quickly learn a target concept like the example of admission criteria whereas other methods may not. We also show that our concept learner can tolerate noise, i.e., it works well even when a target concept is not in $k$-CNF and even when training data contain some errors. In addition, our multiresolution/hierarchical learning approach can drastically reduce learning time and make MEGA effective when it learns a concept in very high-dimensional spaces.

The rest of this paper is organized into six sections. Section 2 depicts the sample selection and feature reduction steps of MEGA. Section 3 describes how MEGA uses a multi-resolution/hierarchical learning method to deal with very large database issues. Section 4 presents our experimental results. Finally, we offer our conclusion in Section 6.

### 1.2 Definitions and Notations

Our learner models query concepts in $k$-CNF and uses $k$-DNF to guide the sampling process.

**Definition 1**: $k$-**CNF**: For constant $k$, the representation class $k$-CNF consists of Boolean formulae of the form $c_1 \wedge \cdots \wedge c_\theta$, where each $c_i$ is a disjunction of at most $k$ literals over the Boolean variables $x_1, \ldots, x_n$. No prior bound is placed on $\theta$.

**Definition 2**: $k$-**DNF**: For constant $k$, the representation class $k$-DNF consists of Boolean formulae of the form $d_1 \vee \cdots \vee d_\theta$, where each $d_i$ is a conjunction of at most $k$ literals over the Boolean variables $x_1, \ldots, x_n$. No prior bound is placed on $\theta$.

In our retrieval system, queries are boolean expressions consisting of predicates connected by the boolean operators $\vee$ (or) and $\wedge$ (and). A predicate on attribute $x_k$ in our system is in the form of $P_{x_k}$. A database system consists of a number of predicates. Our goal is to find the proper operators to combine individual predicates to form the query concept. We choose $k$-CNF to model query concepts, since it can express most practical queries and can be learned via positive-labeled instances in polynomial time [8, 13]. In addition, we use non-positive-labeled instances to refine our sampling space, which we will discuss in detail in Section 2.

A $k$-CNF has three characteristics:

**1**: The terms (or literals) are combined by the $\wedge$ operator.

**2**: The predicates in a term are combined by the $\vee$ operator.

**3**: A term can have at most $k$ predicates.

Suppose we have three predicates $P_{x_1}$, $P_{x_2}$ and $P_{x_3}$. The 2-CNF of these predicates is

$$P_{x_1} \wedge P_{x_2} \wedge P_{x_3} \wedge (P_{x_1} \vee P_{x_2}) \wedge (P_{x_1} \vee P_{x_3}) \wedge (P_{x_2} \vee P_{x_3}).$$

To find objects that are similar to a $k$-CNF concept, we need to measure similarity between objects and the concept. We need to first measure similarity at the predicate level and then at the object level. At the predicate level, we let $F_{x_k}(i, \beta)$ be the distance function that measures the similarity between object $i$ and concept $\beta$ with respect to attribute $x_k$. We normalize the similarity score $F_{x_k}(i, \beta)$ by defining it to be between zero and one. Let $P_{x_k}(i, \beta)$ denote the normalized form. $P_{x_k}(i, \beta) = 0$ means that object $i$ and concept $\beta$ have no similarity with respect to attribute $x_k$, and $P_{x_k}(i, \beta) = 1$ means that the objects with respect to $x_k$ are the same.

Suppose a dataset contains $N$ objects, denoted as $O_i$, where $i = 1 \ldots N$. Suppose each object can be depicted by $M$ attributes, each of which is denoted by $x_k$, where $k = 1 \ldots M$. At the object level, we use the standard fuzzy rules, as defined by Zadeh [4, 21], to aggregate individual predicates' similarity scores. To combine $M$ similarity scores into one aggregated score, we need an *M-ary* aggregation function that maps $[0, 1]^M$ to $[0, 1]$. The rules are as follows:

**Conjunctive rule:** $P_{x_1 \wedge x_2 \wedge \ldots \wedge x_M}(i, \beta) = \min\{P_{x_1}(i, \beta), P_{x_2}(i, \beta), \ldots P_{x_M}(i, \beta)\}$.

**Disjunctive rule:** $P_{x_1 \vee x_2 \vee \ldots \vee x_M}(i, \beta) = \max\{P_{x_1}(i, \beta), P_{x_2}(i, \beta), \ldots P_{x_M}(i, \beta)\}$.

To assist the reader, Table 2 summarizes the parameters that we have introduced and that we will discuss in the rest of the paper.

# 2 The MEGA Algorithm

In this section, we describe how our learning algorithm, MEGA, works. In Section 3, we discuss how MEGA deals with very large database issues such as high dimensional data and very large datasets.

Our learning algorithm consists of three major parts:

- Initialization: Provide users with a reasonable way to convey initial hints to the system.
- Refinement: Refine the query concept based on positive-labeled instances. The refinement step is carefully designed to tolerate noisy data.
- Sampling: Refine the sampling space based on negative-labeled instances and select samples judiciously for expediting the learning process.

## 2.1 Initialization

A user who cannot specify his/her query concept precisely can initially give the concept learner (i.e., MEGA) some hints to start the learning process. For instance, a search for a document or for an image can start with a keyword search or by selecting one or a few categories. We believe this bootstrapping method is more practical than that of most traditional multimedia search engines, which make the unrealistic assumption that users can provide "perfect" examples (i.e., samples) to perform a query. The bootstrapping method presents a set of samples to the user, and the user then labels a set of positive objects. This step gives MEGA a positive-labeled set and a negative-labeled set.

| *Parameter* | *Description* |
|---|---|
| $U$ | Unlabeled dataset |
| $M$ | The number of attributes for depicting a data object |
| $N$ | The number of data objects in $U$ |
| $u$ | A set of samples selected from the unlabeled set $U$ |
| $x_i$ | The $i^{th}$ attribute |
| $O_j$ | The $j^{th}$ object |
| $y_j$ | The label of the $j^{th}$ object |
| $y$ | The labeled set $u$ |
| $y^+$ | The positive-labeled set |
| $y^-$ | The negative-labeled set |
| $QCS$ | The set representation of the query concept space in $k$-CNF |
| $CCS$ | The set representation of the candidate concept space in $k$-DNF |
| $d_i$ | The $i^{th}$ disjunctive term in $QCS$ |
| $c_i$ | The $i^{th}$ conjunctive term in $CCS$ |
| $t_i$ | $d_i$ or $c_i$ |
| $F_{x_k}(i,\beta)$ | Distance measure between $O_i$ and $QCS$ with respect to $x_k$ |
| $P_{x_k}(i,\beta)$ | Normalized $F_{x_k}(i,\beta)$ |
| $P_{t_k}(i,\beta)$ | Normalized $F_{t_k}(i,\beta)$ |
| $P_{t_i\|y_j}$ | The probability of removing term $t_i$ given $y_j$ |
| $P_{t_i\|y}$ | The probability of removing term $t_i$ given $y$ |
| $K_\alpha$ | Sample size |
| $K_c$ | The threshold of eliminating a conjunctive term, $c_i$ |
| $K_d$ | The threshold of eliminating a disjunctive term, $d_i$ |
| $\gamma$ | Voting parameter |
| $f()$ | Func. computing the prob. of removing term $t_i$ given $y_j$ |
| $Vote()$ | Func. computing the aggregated probability of removing $t_i$ |
| $Sample()$ | Sampling func., which selects $u$ from $U$ |
| $Feedback()$ | Labeling function |
| $Collapsed?()$ | The version space has collapsed? true or false |
| $Converged?()$ | The version space has converged? true or false |

Table 2: Parameters.

## 2.2 Refinement

We use Valiant's learning algorithm [19] as the starting point to refine a $k$-CNF concept. We extend the algorithm to

**1**: Handle the fuzzy membership functions (described in Section 1.2),

**2**: Select samples judiciously to expedite the learning process (Section 2.3), and

**3**: Tolerate user errors (Section 2.6).

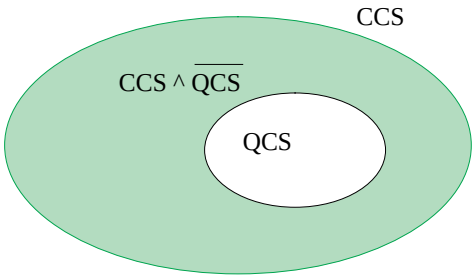



Figure 1: MEGA's Sampling Space: $CCS \wedge \overline{QCS}$.

MEGA initializes the query concept space ($QCS$) as a $k$-CNF and the candidate concept space ($CCS$) as a $k$-DNF. The $QCS$ starts as the most specific concept and the $CCS$ as the most general concept. The target concept that MEGA learns is more general than the initial $QCS$ and more specific than the initial $CCS$. It will become evident shortly that MEGA learns the $QCS$, while at the same time refining the $CCS$ to delimit the boundary of the sampling space. (The shaded area in Figure 1 shows the sampling space between the $QCS$ and the $CCS$).

Figure 2 depicts the learning algorithm. The first step initializes $QCS$ and $CCS$. The second step describes how a query concept is refined through relevance feedback. This refinement step consists of the four substeps shown.

**Step 2.a**: This is MEGA's S-step. MEGA Generates samples from the unlabeled pool $U$. This step passes $QCS$, $CCS$ and $U$ to procedure *Sample* to generate $K_\alpha$ samples. We will discuss procedure *Sample* in Section 2.3.

**Step 2.b**: Solicit user feedback. A user marks an object positive if the object fits his/her query concept. An unmarked object is considered negative.

**Step 2.c**: This is MEGA's feature reduction step. It refines $QCS$ and $CCS$.
Step 2.c.1 refines $QCS$. For each disjunctive term in the $QCS$, MEGA examines each positive-labeled instance and uses function $f$ to compute the probability that the disjunctive term should be eliminated. MEGA then calls procedure *Vote* to tally the votes among the positive-labeled instances and compares the vote with threshold $K_d$ to decide whether that disjunctive term is to be removed. If sufficient numbers of positive-labeled instances contradict the $QCS$ with respect to a disjunctive term (i.e., if the threshold is exceeded), the term is removed from the $QCS$. We discuss procedure *Vote*, which decides how aggressive MEGA is in eliminating terms, in Section 2.6.
Step 2.c.2 refines $CCS$. Similar to the steps in 2.c.1, for each conjunctive term in the $CCS$, MEGA examines each negative-labeled instance, and uses function $f$ to compute the probability that the conjunctive term should be eliminated. MEGA then calls procedure *Vote* to tally the votes among the negative-labeled instances. Then it compares the vote with threshold $K_c$ to decide whether that conjunctive term is to be removed. If sufficient numbers of negative-labeled instances satisfy the $CCS$ with respect to a conjunctive term, the term is removed from the $CCS$.

**Step 2.d**: Perform bookkeeping by reducing the unlabeled pool.

The refinement step terminates when the learning process converges to the target concept ($Converged? = true$) or the concept is collapsed ($Collapsed? = true$). ($Converged?$ and $Collapsed?$ are defined below.) In practice, the refinement stops when no unlabeled instance $u$ can be found between the $QCS$ and the $CCS$.

**Definition 3:** $Converged?(QCS, CCS)$

$$Converged?(QCS, CCS) \leftarrow true \ if \ CCS == QCS; \quad false \ otherwise.$$

**Definition 4:** $Collapsed?(QCS, CCS)$

$$Collapsed?(QCS, CCS) \leftarrow true \ if \ CCS \in QCS; \quad false \ otherwise.$$

**Algorithm MEGA**

*Input*: $U$, $K_c$, $K_d$, $K_\alpha$;

*Output*: $QCS$;

*Procedure calls*: $f()$, $Vote()$, $Sample()$, $Feedback()$, $Collapsed?()$, $Converged?()$;

*Variables*: $u$, $y$, $U$, $P_{t_i|y_j}$, $P_{t_i|y}$;

**Begin**

**1** Initialize the version space
$QCS \leftarrow \{d_1, d_2, \ldots\}$; $CCS \leftarrow \{c_1, c_2, \ldots\}$;

**2** Refine query concept via relevance feedback
While ($not\ Collapsed?(QCS, CCS)\ and\ not\ Converged?(QCS, CCS)$)

**2.a** S-step: sample selection
$u \leftarrow Sample(QCS, CCS, U, K_\alpha)$;

**2.b** Solicit user feedback
For each $u_i \in u$
$y_i \leftarrow Feedback(u_i)$;

**2.c** F-step: feature reduction

**2.c.1** Refine $k$-CNF using positive samples
For each $d_i \in QCS$
For each $y_j \in y^+$
$P_{d_i|y_j} \leftarrow f(d_i, O_j, QCS)$;
$P_{d_i|y^+} \leftarrow Vote(y^+, P_{d_i|y_j \in y^+}, \gamma)$;
If ($P_{d_i|y^+} > K_d$)
$QCS \leftarrow QCS - \{d_j\}$;

**2.c.2** Refine $k$-DNF using negative samples
For each $c_i \in CCS$
For each $y_j \in y^-$
$P_{c_i|y_j} \leftarrow f(c_i, O_j, CCS)$;
$P_{c_i|y^-} \leftarrow Vote(y^-, P_{c_i|y_j \in y^-}, \gamma)$;
If ($P_{c_i|y^-} > K_c$)
$CCS \leftarrow CCS - \{c_j\}$;

**2.d** Bookkeeping
$U \leftarrow U - u$;

**3** Return query concept
Output $QCS$;

**End**

Figure 2: Algorithm MEGA.

## 2.3 Sampling

MEGA invokes procedure *Sample* to select the next $K_\alpha$ unlabeled instances[3] to ask for user feedback. From the college-admission example presented in Section 1, we learn that if we would like to minimize our work (i.e., call a minimum number of friends), we should choose our samples judiciously. But, what constitutes a good sample? We know that we learn nothing from a sample if

- It agrees with the concept in all terms.
- It has the same attributes as another sample.
- It is unlikely to be labeled positive.

To make sure that a sample is useful, MEGA employs two strategies:

**1**. Bounding the sample space: MEGA avoids choosing useless unlabeled instances by using the $CCS$ and $QCS$ to delimit the sampling boundary.

**2**. Maximizing the usefulness of a sample: MEGA chooses a sample that shall remove the maximum expected number of disjunctive terms. In other words, MEGA chooses a sample that can maximize the expected generalization of the concept.

It may appear that if we pick a sample that has more dissimilar disjunctions (compared to the $QCS$), we may have a better chance of eliminating more disjunctive terms. This is, however, not true. A sample must be labeled by the user as positive to be useful for refining $QCS$. Unfortunately, a sample with more disjunctions that are dissimilar to the target concept is less likely to be labeled positive. Therefore, in choosing a sample, there is a tradeoff between those with more contradictory terms and those more likely to be labeled positive.

Let $\Psi$ denote the number of disjunctions remaining in the concept. The number of disjunctions that can be eliminated in the current round of sampling (denoted as $\psi$) is between zero and $\Psi$. We can write the probability of eliminating $\psi$ terms as $P_e(\psi)$. $P_e(\psi)$ is a monotonically decreasing function of $\psi$.

Our MEGA scheme attempts to find the $\psi$ that can eliminate the maximum expected number of disjunctive terms, given a sample. MEGA's objective function can be written as

$$\psi^* = argmax_\psi E(\psi) = argmax_\psi(\psi \times P_e(\psi)). \quad (1)$$

To solve $\psi^*$, we must know $P_e(\psi)$, which can be estimated by the two methods below: *probabilistic estimation* and *empirical estimation*.

[3] In this study we focus our discussion on sample selection criteria rather than on access methods for finding samples on disk. We are currently enhancing a high-dimensional indexing scheme that we have developed [12] to achieve efficient sample retrieval.

## 2.4 Probabilistic Estimation

We first consider how to estimate $\psi^*$ using a probability model. As we have seen in the college-admission example, if a sample contradicts more disjunctive terms, it is more likely to be labeled negative (i.e., less likely to be labeled positive). For example, a sample that contradicts predicate $P_1$ is labeled negative only if $P_1$ is in the user's query concept. A sample that contradicts both predicates $P_1$ and $P_2$ is labeled negative if either $P_1$ or $P_2$ is in the user's query concept.

Formally, let random variable $\Phi_i$ be 1 if $P_i$ is in the concept and 0 otherwise. For simplicity, let us assume that the $\Phi_i$'s are iid (independent and identically distributed), and the probability of $\Phi_i$ being 1 is $p$ $(0 < p < 1)$. The probability of a sample contradicting $\psi$ disjunctive terms is marked positive only when none of these $\psi$ terms appears in the user's query concept. This probability is $(1-p)^\psi$. If we substitute $P_e(\psi)$ by $(1-p)^\psi$ on the right-hand side of Equation 1, we get

$$\max E(\psi) = \psi(1-p)^\psi.$$

If we take the derivative of $E(\psi)$, we can find the optimal $\psi$ value, denoted by $\psi^*$:

$$\psi^* = \Psi,\ if\ \frac{1}{ln\frac{1}{1-p}} > \Psi, \quad \psi^* = \frac{1}{ln\frac{1}{1-p}},\ otherwise.$$

Of course, it may be too strong an assumption that the probability $p$ of all disjunctions is iid. However, we do not need a precise estimation here for the following two reasons:

1. Precise estimation may not be feasible and can be computationally intensive.
2. An approximate estimation is sufficient for bootstrapping. Once the system is up and running for a while and collects enough data, it can empirically estimate $P_e(\psi)$ using its past experience. We discuss this process next.

## 2.5 Empirical estimation

MEGA can estimate $P_e(\psi)$ based on its past experience. For each sample it presents, it records how many disjunctions the sample contradicts with respect to the query concept and whether the sample is labeled positive. Once the system collects a sufficient amount of data, we can estimate $P_e(\psi)$ empirically. We then pick the $\psi^*$ that can eliminate the maximum expected number of disjunctive terms.

Again, a reasonable approach to estimate $P_e(\psi)$ is to use probabilistic estimation when the system first starts and then switches to empirical estimation when the system has collected sufficient data. In such a case there can be an abrupt transition when MEGA switches from one estimation to the other, since the two estimates of $P_e(\psi)$ may differ substantially. This may lead to a sudden change in behavior of the sampling component of the active learner. To remedy this problem, we employ a Bayesian smoothing approach. Essentially the probabilistic estimation is the prior guess at the

distribution over $\psi$ and the empirical approach is the guess based purely on the data that MEGA has seen so far. The Bayesian approach combines both of these guesses in a principled manner. Before we start, we imagine that we have seen a number of samples of $\psi$. After refinement iteration we gather new samples for $\psi$; then we add them to our current samples and adjust $P_e(\psi)$.

For example, before we start, we assume that we have already seen samples with $\psi = 1$ being labeled positive three out of five times and samples with $\psi = 2$ being labeled positive seven out of 20 times. In other words, we have successfully eliminated $\psi = 1$ term three times out of five, and we have successfully eliminated $\psi = 2$ terms 7 times out of 20. Thus initially $P_e(\psi = 1) = 3/5 = 0.6$ and $P(\psi = 2) = 7/20 = 0.35$. Now suppose we do a query and in which we observe a sample with $\psi = 2$ being labeled positive. Then our new distribution is $P(\psi = 1) = 3/5$ and $P(\psi = 2) = 8/21$. We continue in this manner. At first the prior assumption has quite an effect on our guess about the distribution. The more imaginary samples we have in our prior assumption, the larger its effect. For instance, if we assume that $\psi = 1$ being labeled positive 30 out of 50 times and that $\psi = 2$ being labeled positive 70 out of 200 times, it takes more real samples to change $P_e(\psi)$. With time, the more real samples we get, the less the effect of the prior assumption becomes, until eventually it has virtually no effect, and the observed data dominate the expression. This procedure gives us a smooth transition between the "probabilistic" and the "empirical" methods.

## 2.6 Procedure Vote

In the previous example, we assume that all samples are noise-free. This assumption may not be realistic. There can be two sources of noise:

- Model bias: The target concept may not be in $k$-CNF.
- User errors: A user may label some positive instances negative and vice versa.

**Procedure Vote**
*Input*: $y$, $P_{t_i|y_j \in y}$, $\gamma$;
*Output*: $P_{t_i|y}$;
**Begin**
Sort $P_{t_i|y_j}$ in the descending order;
Return the $\gamma^{th}$ highest $P_{t_i|y}$;
**End**

Figure 3: Procedure Vote.

Procedure *Vote* controls the strictness of voting using $\gamma$. The larger the value of $\gamma$ is, the more strict the voting is and therefore the harder it is to eliminate a term. When the noise level is high, we have less confidence on the correctness of user feedback. Thus, we want to be more cautious about eliminating a term. Being more cautious means increasing $\gamma$. Increasing $\gamma$, however, makes the learning process converge more slowly. To learn a concept when noise is present, one has to buy

accuracy with time.

# 3 Multi-resolution/Hierarchical Learning

The MEGA scheme described so far does not yet concern its scalability with respect to $M$ (the number of features for depicting an object). In this section, we describe MEGA's multiresolution/hierarchical learning algorithm that tackles the *dimensionality-curse* problem.

The number of disjunctions in a $k$-CNF (and, likewise, the conjunctives in a $k$-DNF) can be written as

$$\sum_{i=1}^{k} \binom{M}{i}. \tag{2}$$

When $M$ is large, a moderate $k$ can result in a large number of disjunctive terms in a $k$-CNF, which causes high space and time complexity for learning. For instance, an image database that we have built [1] characterizes each image with 144 features ($M = 144$). The initial number of disjunctions in a 3-CNF is half a million and in a 4-CNF is eighteen million.

To reduce the number of terms in a $k$-CNF, we divide a learning task into $G$ sub-tasks, each of which learns a subset of the features. Dividing a feature space into $G$ subspaces reduces both space and time complexity by a factor of $O(G^{k-1})$. For instance, setting $G = 12$ in our image database reduces both space and time complexity for learning a 3-CNF by 140 times (the number of terms is reduced to $3,576$), and for learning a 4-CNF by $1,850$ times (the number of terms is reduced to $9,516$). The savings is enormous in both space and learning time. (The wall-clock time is less than a second for one learning iteration for a 4-CNF concept on a Pentium-III processor.)

This divide-and-conquer approach may trade precision for speed, since some terms that involve features from more than one feature subset can no longer be included in a concept. The loss of precision can be reduced by organizing a feature space in a multi-resolution fashion. The term *feature resolution* and a weak form of feature resolution that we call *feature correlation* are defined as follows:

**Definition 5**: **Feature resolution**: Feature $P_i$ is said to have higher resolution than feature $P_j$ if the presence of $P_i$ implies the presence of $P_j$ (or the absence of $P_j$ implies the absence of $P_i$). Let $P_i \in P_j$ denote that $P_i$ has higher resolution than $P_j$. We say that $P_i \in P_j$ if and only if the conditional probability $P(P_j|P_i) = 1$.

**Definition 6**: **Feature correlation**: A feature $P_i$ is said to have high correlation with feature $P_j$ if the presence of $P_i$ implies the presence of $P_j$ and vice versa with high probability. We say that $P_i \sim P_j$ if and only if the conditional probability $P(P_j|P_i)/P(P_j) = P(P_i|P_j)/P(P_i) \geq \delta$.

MEGA takes advantage of feature resolution and correlation in two ways— *inter-group* multi-resolution and *intra-group* multi-resolution —for achieving fast and accurate learning. Due to the space limitation, we limit our description of the heuristics of MEGA's multi-resolution learning algorithm to the following.

- Inter-group multi-resolution features. If features can be divided into groups of different resolutions, we do not need to be concerned with terms that involve inter-group features. This is because any inter-group terms can be subsumed by intra-group terms. Formally, if $P_i$ and $P_j$ belong to two feature groups and $P(P_i|P_j) = 1$, then $P_1 \vee P_2 = P_2$ and $P_1 \wedge P_2 = P_1$.
- Intra-group multi-resolution features. Within a feature group, the more predicates involved in a disjunctive term, the lower the resolution of the term. Conversely, the more number of predicates involves in a conjunctive term, the higher resolution the term is. For instance, in a 2-CNF that has two predicates $P_1$ and $P_2$, term $P_1$ and term $P_2$ have a higher resolution than the disjunctive term $P_1 \vee P_2$ and a lower resolution than the conjunctive term $P_1 \wedge P_2$. The presence of $P_1$ or $P_2$ makes the presence of $P_1 \vee P_2$ useless. Base on this heuristic, MEGA examines a term only when all its higher resolution terms have been eliminated.

# 4 Experiments

In this section we report our experimental results. The goals of our experiments were

**1**: To evaluate whether MEGA can learn $k$-CNF concepts accurately in the presence of a large number of irrelevant features.

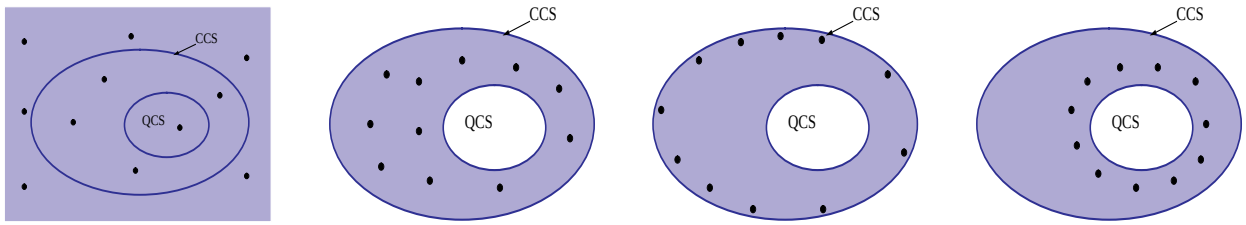
**2**: To evaluate whether MEGA can converge to a target concept faster than traditional sampling schemes.

**3**: To evaluate whether MEGA is robust for noisy data or under situations in which the unknown target concept is not expressible in the provided hypothesis space.

We assume all target concepts are in 3-CNF. To conduct our experiments, we used both synthesized data and real-world data.

- Synthesized data. We generated three datasets using two different distributions: uniform and Gaussian. Each instance has 10 features between 0 and 1. The values of each feature in a dataset are independently generated. For the Gaussian distribution, we set its mean to 0.5 and its standard deviation to 1/6. Each dataset has 10,000 vectors.
- Real-world data. We conducted experiments on a 1,500-image dataset collected from Corel image CDs and the Internet. The images in the dataset belong to ten categories—*architecture*, *bears*, *clouds*, *flowers*, *landscape*, *people*, *objectionable images*, *tigers*, *tools*, and *waves*. Each image is characterized by a 144 dimensions feature vector (described in Section 4.3).

We used *precision* and *recall* to measure performance. We tallied precision/recall for up to only ten iterations, since we deemed it unrealistic to expect an interactive user to conduct more than ten rounds of relevance feedback. We compared MEGA with the five sampling schemes: *random*, *bounded random*, *nearest neighbor*, *query expansion* and *aggressive*. We used these sampling schemes for comparison because they are employed by some state-of-the-art systems described in Section 5.




(a) Random (b) B-Random (c) Aggressive (d) MEGA

Figure 4: Sampling Schemes.

Figure 4 shows how some of these sampling algorithms work. The main features of the sampling schemes are given below.

- *Random*: Samples are randomly selected from the bulk of the domain (Figure 4(a)).
- *Bounded Random*: Samples are randomly selected from between $QCS$ and $CCS$ (Figure 4(b)).
- *Nearest Neighbor*: Samples are selected from the nearest neighborhood of the center of the positive-labeled instances.
- *Query Expansion*: Samples are selected from the neighborhood of multiple positive-labeled instances.
- *Aggressive*: Samples are selected from the unlabeled ones that satisfy the most general concepts in $CCS$ (Figure 4(c)).
- *MEGA*: Samples are selected between $QCS$ and $CCS$ to eliminate the maximum expected number of terms (Figure 4(d)).

We ran experiments on datasets of different distributions and repeated each experiment ten times. The experimental results are presented in two groups. We first show the results of the experiments on the synthesized datasets. We then show the results on a $1,500$-image databset.

## 4.1 Query Concept Learning Applied to Synthesized Datasets

We tested many target concepts on the two synthesized datasets. Due to space limitations, we present only three representative test cases, those that represent a disjunctive concept, a conjunctive of disjunctions, and a complex concept with more terms. The three tests are

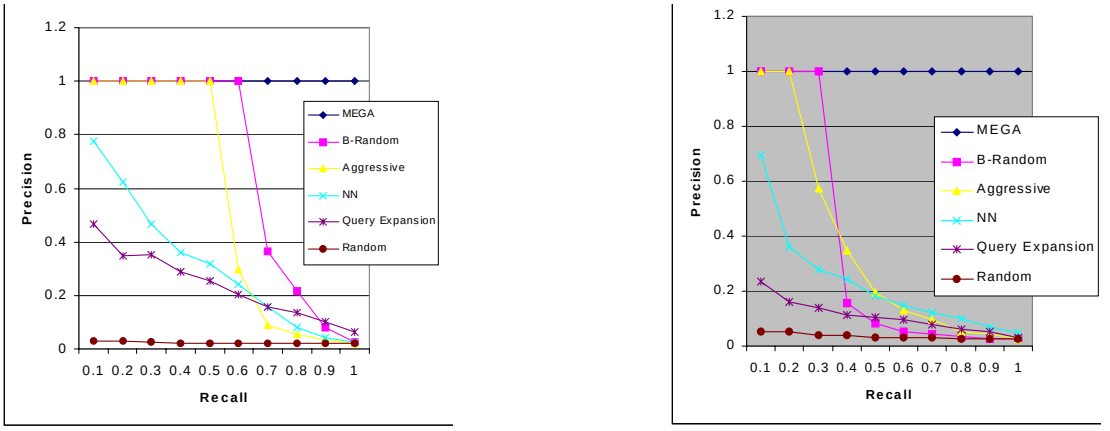
**1**: $P_1 \vee P_2$,

**2**: $(P_1 \vee P_2) \wedge P_3$,

**3**: $P_1 \wedge (P_2 \vee P_3) \wedge (P_4 \vee P_5 \vee P_6) \wedge (P_2 \vee P_4 \vee P_7)$,

We first assume that the dataset is free of user errors and set the sample size $K_\alpha$ to 20. In the remainder of this section, we report our initial results, and then we report the effects of model bias and user errors on MEGA (Sections 4.2.1 and 4.2.2).

### 4.1.1 Experimental Results




(a) $(P_1 \vee P_2) \wedge P_3$ (b) Complex

Figure 5: Precision vs. Recall (10 Features).

Figure 5 presents the precision/recall after three user iterations of the six sampling schemes learning the two concepts, $(P_1 \vee P_2) \wedge P_3$ and $P_1 \wedge (P_2 \vee P_3) \wedge (P_4 \vee P_5 \vee P_6) \wedge (P_2 \vee P_4 \vee P_7)$. The performance trend of the six schemes is similar at different numbers of iterations. We deem three iterations a critical juncture where a user would be likely to lose his/her patience, and thus we first present the results at the end of the third iteration. The performance curve of MEGA far exceeds that of the other five schemes at all recall levels. Note that for learning both concepts, MEGA achieves 100% precision at all recall levels.

Next, we were interested in learning the improvement on search accuracy with respect to the number of user iterations. This improvement trend can tell us how fast a scheme can learn a target concept. We present a set of tables and charts where we fix recall at 0.5 and examine the improvement in precision with respect to the number of iterations.

| *Rnd #* | *Random* | *B-Random* | *N-Neighbor* | *Q-Expansion* | *Aggressive* | *Algorithm MEGA* |
|---|---|---|---|---|---|---|
| 1 | 0.23715 | 0.23715 | 0.20319 | 0.20319 | 0.23715 | 0.23715 |
| 2 | 0.44421 | 0.44421 | 0.48207 | 0.44422 | 0.44421 | 0.30098 |
| 3 | 0.49507 | 0.50389 | 0.41036 | 0.45219 | 0.50389 | 1.00000 |
| 4 | 0.50389 | 1.00000 | 0.36753 | 0.51394 | 1.00000 | 1.00000 |
| 5 | 1.00000 | 1.00000 | 0.35857 | 0.78088 | 1.00000 | 1.00000 |
| 6 | 1.00000 | 1.00000 | 0.33865 | 0.88247 | 1.00000 | 1.00000 |
| 7 | 1.00000 | 1.00000 | 0.32669 | 0.93028 | 1.00000 | 1.00000 |
| 8 | 1.00000 | 1.00000 | 0.32271 | 0.93028 | 1.00000 | 1.00000 |
| 9 | 1.00000 | 1.00000 | 0.29880 | 0.93028 | 1.00000 | 1.00000 |
| 10 | 1.00000 | 1.00000 | 0.32570 | 0.93028 | 1.00000 | 1.00000 |

Table 3: Learning $P_1 \vee P_2$ Applied to A Uniform Dataset.

Tables 3 and 4 present the precision of six sampling schemes in learning $P_1 \vee P_2$ in ten rounds of relevance feedback. These tables show that MEGA consistently converges to the target concept in the

smallest number of iterations. Applied to the Gaussian dataset, MEGA converges after four iterations. The random sampling scheme requires on average two more iterations to converge. The performance of the bounded random scheme and the performance of the aggressive scheme fall between that of the random scheme and that of MEGA. On the aggressive scheme, which attempts to remove as many terms as possible, the chosen samples are less likely to be labeled positive and hence make less of a contribution to the progress of learning the $QCS$. We will show shortly that the gaps in performance between MEGA and the other schemes increase as the target concept becomes more complex.

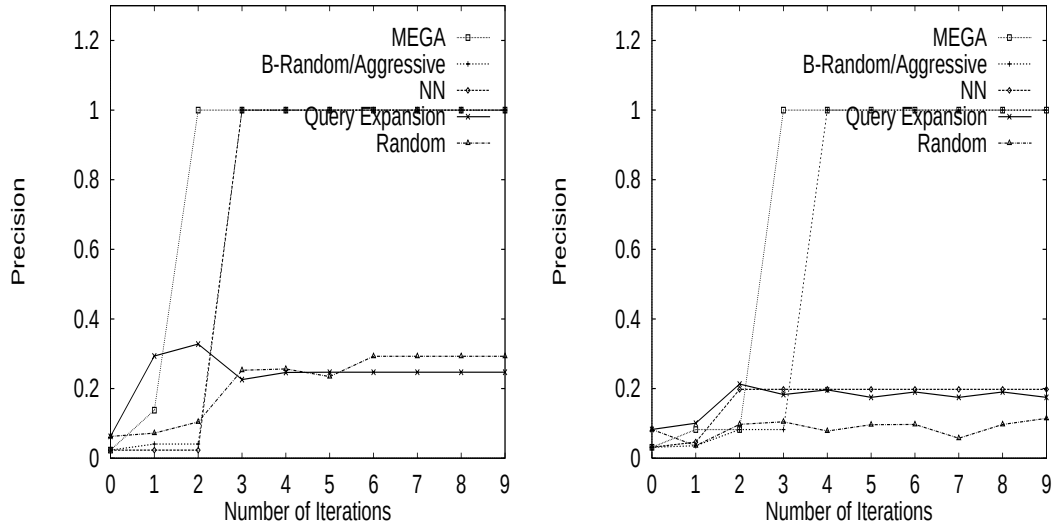
| *Rnd #* | *Random* | *B-Random* | *N-Neighbor* | *Q-Expansion* | *Aggressive* | *Algorithm MEGA* |
|---|---|---|---|---|---|---|
| 1 | 0.08236 | 0.08236 | 0.29970 | 0.29970 | 0.08236 | 0.08236 |
| 2 | 0.22178 | 0.22178 | 0.65722 | 0.46684 | 0.36241 | 0.32438 |
| 3 | 0.37332 | 0.37332 | 0.64907 | 0.47027 | 0.80584 | 0.65982 |
| 4 | 0.38200 | 0.51249 | 0.64134 | 0.46598 | 0.80584 | 1.00000 |
| 5 | 0.51249 | 1.00000 | 0.63941 | 0.66237 | 0.80584 | 1.00000 |
| 6 | 1.00000 | 1.00000 | 0.62782 | 0.46491 | 0.80584 | 1.00000 |
| 7 | 1.00000 | 1.00000 | 0.61000 | 0.47135 | 0.80584 | 1.00000 |
| 8 | 1.00000 | 1.00000 | 0.61000 | 0.61258 | 0.80584 | 1.00000 |
| 9 | 1.00000 | 1.00000 | 0.61000 | 0.48830 | 0.80584 | 1.00000 |
| 10 | 1.00000 | 1.00000 | 0.61000 | 0.64198 | 0.80584 | 1.00000 |

Table 4: Learning $P_1 \vee P_2$ Applied to Gaussian Dataset.

The results of all datasets and all subsequent tests show that both the nearest neighbor and the query expansion schemes converge very slowly. The result is consistent with that reported in [16, 18], which shows that the query expansion approach does better than the nearest neighbor approach but both suffer from slow convergence. Sampling in the nearest neighborhood tends to result in low precision/recall if the initial query samples are not perfect.

The precision at a given recall achieved by the experiments applied to the Gaussian dataset is lower than that of the experiments applied to the uniform dataset. This is because when an initial query point falls outside of, say, two times the standard deviation, we may not find enough positive examples in the unlabeled pool to eliminate all superfluous disjunctions. Since this situation is rare, the negative effect on the average precision/recall is insignificant. The performance gaps between the six sampling schemes were similar when we applied them to the two datasets; therefore, we report only the results of the experiments on the uniform dataset in the remainder of this section.

Figure 6 depicts the results of the second and third tests on the uniform dataset. The figure shows that MEGA outperforms the other scheme (in precision at a fixed recall) by much wider margins. It takes MEGA only three iterations to learn these concepts, whereas the other schemes progress more slowly. Schemes like nearest-neighbor and query expansion fail miserably because they suffer from severe model bias. Furthermore, they cannot eliminate irrelevant features quickly.

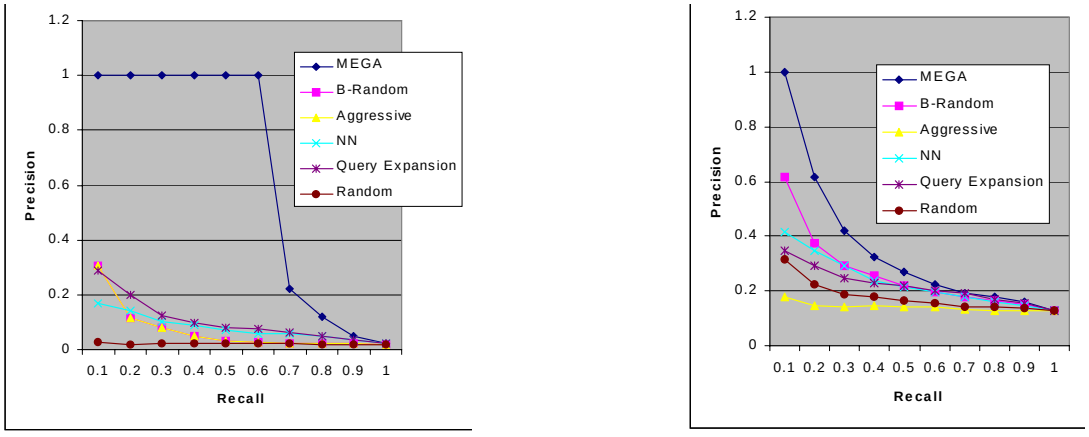



(a) $(P_1 \vee P_2) \wedge P_3$ (b) *Complex Query*

Figure 6: Precision of Six Schemes at Recall = 50%.

## 4.2 Addition Results

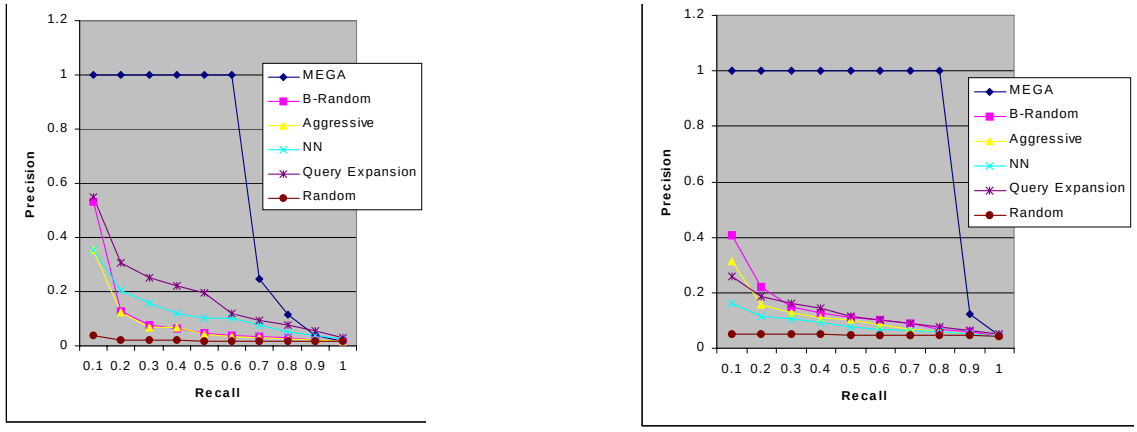
We also performed tests on a 20 and 30 feature dataset. The results are shown in Figures 7 and 8. The higher the dimension, the wider the performance gap between MEGA and the rest of the schemes. This is because MEGA can eliminate irrelevant features much faster than the other schemes.




(a) $(P_1 \vee P_2) \wedge P_3$ (b) Complex

Figure 7: Precision vs. Recall (20 Features).

### 4.2.1 Model Bias Test




(a) $(P_1 \vee P_2) \wedge P_3$ (b) Complex

Figure 8: Precision vs. Recall (30 Features).

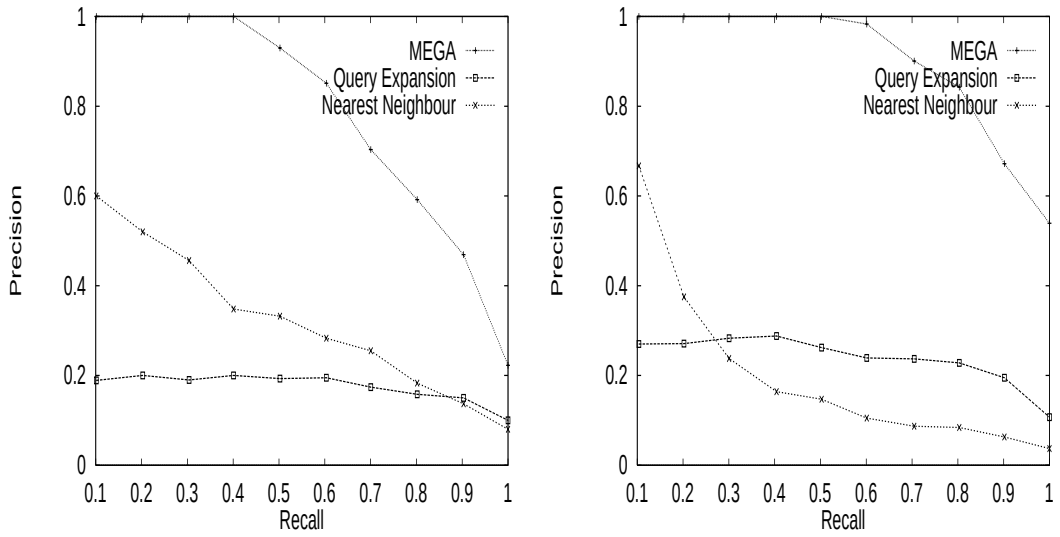
We have shown that MEGA outperforms the other five sampling schemes significantly when the target query concept is in $k$-CNF. We now present test cases that favor a convex concept, which can be expressed as a linear weighted sum of features to examine how MEGA performs. The target concept we tested is in the form of $\alpha P_1 + (1 - \alpha)P_2$, where the value of $\alpha$ is between zero and one.

In this set of tests, we compare MEGA with the nearest neighbor scheme and the query expansion scheme, which are the representative schemes designed for refining convex concepts. We started by picking 20 random images to see how fast each scheme would converge to the target concepts. Again, we repeated each experiment 100 times and recorded each scheme's average precision and recall.

We tested six convex concepts by setting $\alpha = 0, 0.1, \ldots, 0.5$. Below, we report the precision/recall of the three learning methods on two concepts: $0.3P_1 + 0.7P_2$ ($\alpha = 0.3$) and $0.5P_1 + 0.5P_2$ ($\alpha = 0.5$). Setting $\alpha$ in this range makes MEGA suffer from model bias. (We will discuss the reasons shortly.) Figure 9 presents the precision/recall of the three schemes for learning these two concepts after three user iterations. Surprisingly, even though MEGA is not modeled after a convex concept, the performance curve of MEGA far exceeds that of the other two schemes in learning both concepts.

To understand the reasons why MEGA works better than the nearest neighbor and query expansion schemes and how each scheme improves from one iteration to another, we present a set of charts where we fix recall at 0.5 and examine the trend of precision with respect to the number of iterations. (The trend at other recall levels is similar.) Figure 10(a) shows the result of learning concept $P_2$ (setting $\alpha = 0$). MEGA does very well in this experiment, since it suffers no model bias. Neither the nearest neighbor nor the query expansion scheme does as well because they are slow in eliminating terms.

What if a user does have a weighted linear query-concept? Even so, MEGA can approximate this model fairly well. Figures 10(b), (c), (d), (e), and (f) all show that MEGA achieves higher precision faster than either the nearest neighbor or the query expansion scheme under all $\alpha$ settings. We summarize our observations as follows:




(a) $\alpha = 0.3$ (b) $\alpha = 0.5$

Figure 9: Recall vs. Precision (Model Bias Test).

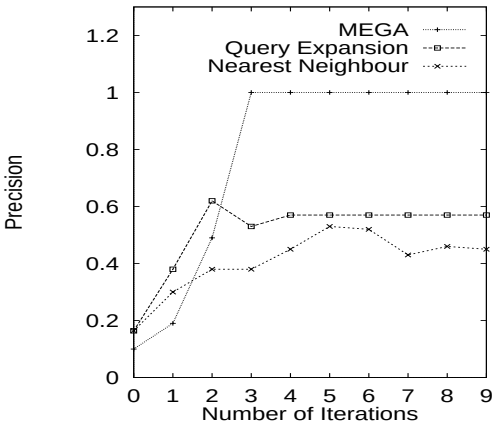
1. When $\alpha = 0$ (or 1), the concept has only one predicate and MEGA has better precision by a wide margin than these traditional schemes, since it can converge much faster. Even when $\alpha$ is near 0 or 1, the precision of MEGA decreases slightly but still outperforms the traditional schemes, as shown in Figure 10(b). This is because although MEGA suffers slightly from model bias, its fast convergence makes it a better choice when the number of iterations is relatively small.

2. When $\alpha = 0.5$, MEGA can approximate the convex concept by $P_1 \wedge P_2$. Figures 10(e) and (f) show that when $\alpha$ is near 0.5, MEGA trails the query expansion by only a slim margin after five/six user iterations. Although the query expansion scheme eventually converges to the target concept, MEGA's fast improvement in precision in just a couple of iterations makes it more attractive, even though slower learning schemes might eventually achieve a slightly higher precision.

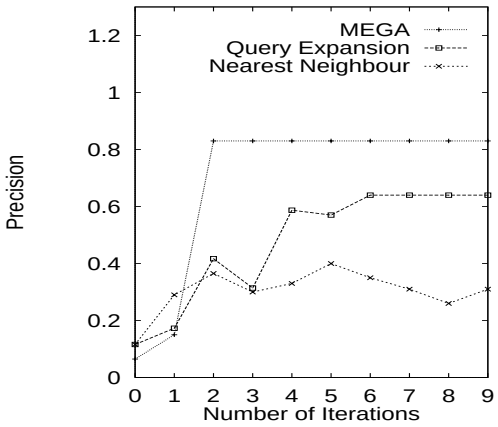
3. Figures 10(c) through (e) show that when $\alpha$ is between 0.2 and 0.4, MEGA suffers from model bias and its achievable precision can be low. However, our primary concern is with the range between three and five iterations that will probably reflect the patience of on-line users. For this purpose, MEGA is more attractive even with its model bias. When $\alpha = 0.2$, MEGA reaches 70% precision after two iterations whereas the query expansion scheme requires seven iterations to reach the same precision.

#### 4.2.2 User Error Test

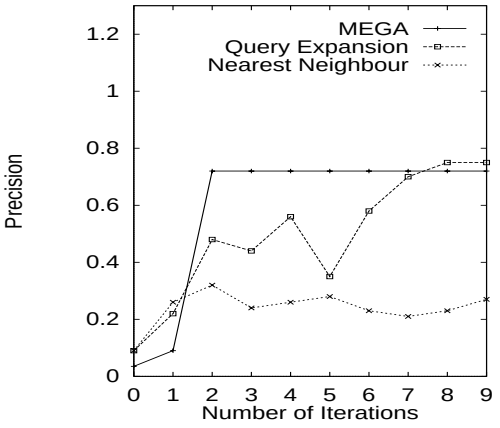
In this experiment we learned the $(P_1 \vee P_2) \wedge (P_3 \vee P_4)$ concept under three different error rates, 5%, 10% and 15%. (A five percent error rate means that one out of twenty samples is mis-labeled.)



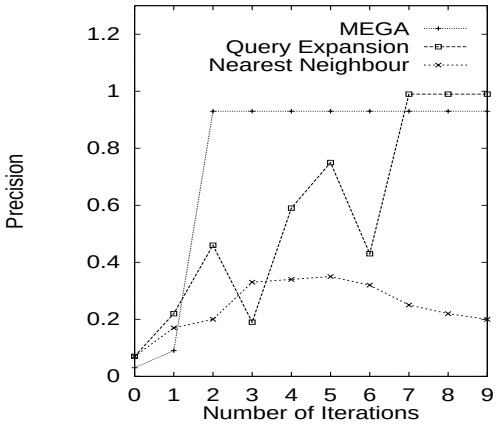

(a) $\alpha = 0(1.0)$



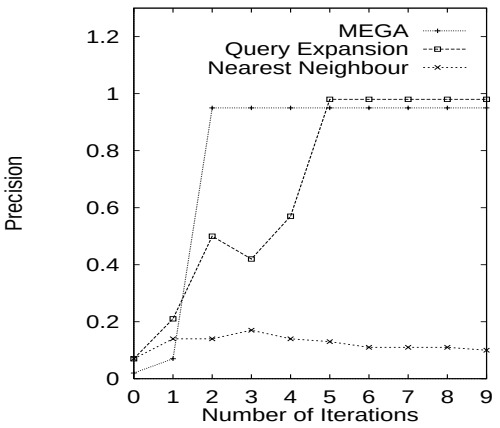

(b) $\alpha = 0.1(0.9)$



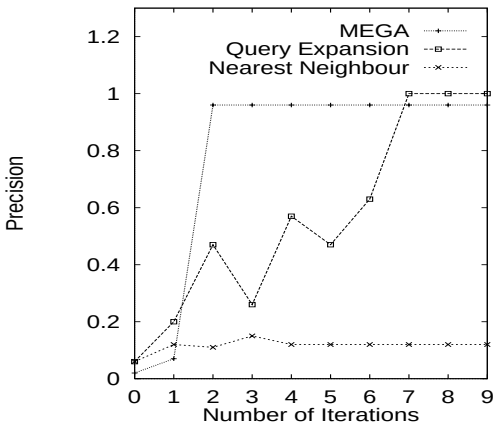

(c) $\alpha = 0.2(0.8)$




(d) $\alpha = 0.3(0.7)$

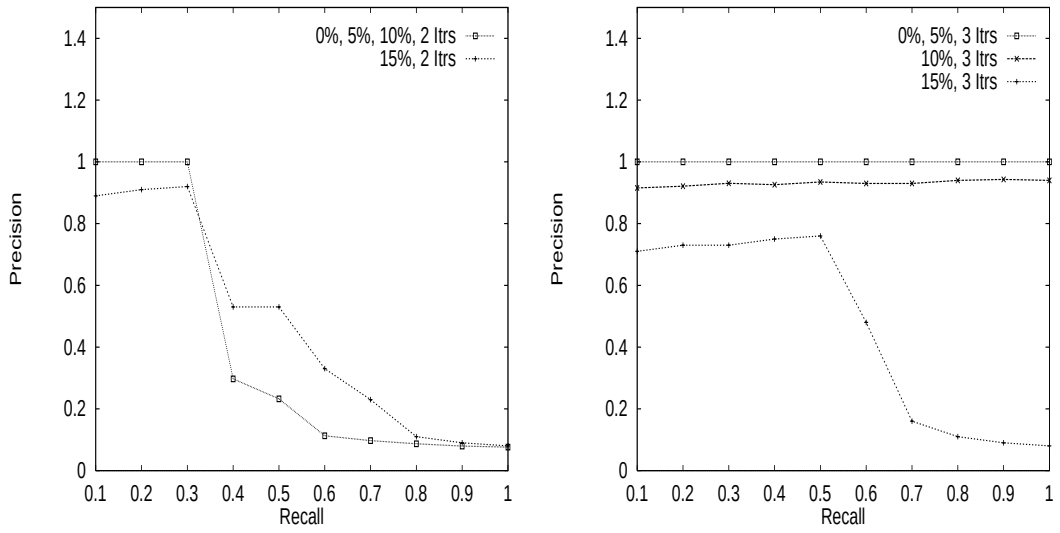



(e) $\alpha = 0.4(0.6)$




(f) $\alpha = 0.5$

Figure 10: The Effect of Different $\alpha$'s.




(a) Two Iterations (b) Three Iterations

Figure 11: Precision/Recall Under 0%, 5%, 10% and 15% Noise.

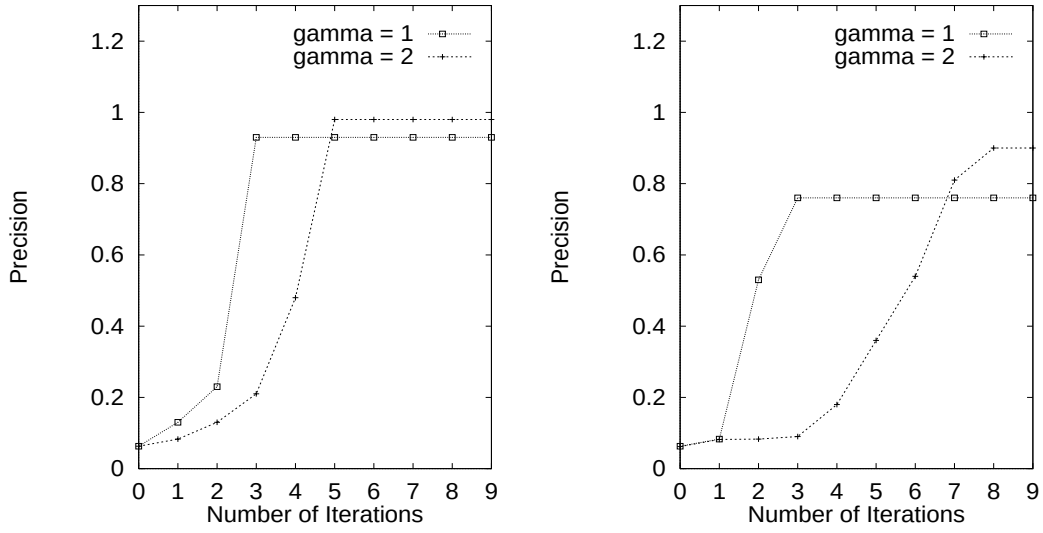
We also used two different $\gamma$ settings (one and two) to examine the tradeoff between learning speed and accuracy. Figure 11 presents the precision/recall after two or three user iterations under different error rates. MEGA enjoys little to no performance degradation when the noise rates are less than or equal to 10%. When the error rate is 15%, MEGA's search accuracy starts to deteriorate. This experiment shows that MEGA is able to tolerate mild user errors.

Next, we fix recall at 50% and examine how different error rates and $\gamma$ settings affect learning precision. Figure 12(a) shows that under both $\gamma = 1$ and $\gamma = 2$ settings, MEGA reaches high precision. However, MEGA's precision improves much faster when $\gamma = 1$ than when $\gamma = 2$. This result does not surprise us, since a lower $\gamma$ value eliminates terms more aggressively and hence leads to faster convergence. When the noise level is high (15%), Figure 12(b) shows that a low $\gamma$ setting hinders accurate learning of the target concept. This is because MEGA eliminates terms too aggressively, and the high noise level causes it to eliminate wrong terms. But if we set $\gamma = 2$, we can learn the concept with higher accuracy by slowing down the learning pace. This experiment shows a clear tradeoff between learning accuracy and convergence speed. When the noise level is low, it is preferable to use a less strict voting scheme (i.e., setting a smaller $\gamma$) for achieving faster convergence. When the noise level is high, a stricter voting scheme (i.e., using a larger $\gamma$) will better maintain high accuracy.

### 4.2.3 Observations

We can summarize the above experimental results as follows:

1. Convergence speed: MEGA converges much faster than the other schemes in all cases.




(a) 10% (b) 15%

Figure 12: Effects of Noise.

2. Model accuracy: MEGA outperforms the other schemes by a wide margin when the target query concept is in $k$-CNF. Even when a user's query concept is a weighted linear function, MEGA can approximate it fairly well. The fact that MEGA can achieve a high convergence ratio in a small number of iterations makes it an attractive on-line learning scheme.

3. Noise tolerance: MEGA does well under noise conditions, including model bias and user errors.

### 4.3 MEGA Applied to An Image Dataset

We also conducted experiments on a 1,500-image dataset [1]. A 144-dimension feature vector was extracted for each image containing information about color histograms, color moments, textures, etc [2]. We divided features into nine sets based on their resolutions (depicted in Table 5). We assumed that query concepts could be modeled in 3-CNF. Each of the query concepts we tested belongs to one of the ten image categories: *architecture*, *bears*, *clouds*, *flowers*, *landscape*, and *people*. *objectionable images*, *tigers*, *tools*, and *waves*. MEGA learned a target concept solely in the feature space and had no knowledge about these categories.

In each experiment, we began with a set of 20 randomly generated images for querying user feedback. After each iteration, we evaluated the performance by retrieving top-$K$ images based on the concept we had learned. We recorded the ratio of these images that satisfied the user's concept. We ran each experiment through up to five rounds of relevance feedback, since we deemed it unrealistic to expect an interactive user to conduct too many rounds of feedback. We ran each experiment ten times with different initial starting samples.

Table 6 shows the precision of the ten query concepts—for $K = 10$ or 20. (Recall is not presented

| Feature Group # | Filter Name | Resolution | Representation |
|---|---|---|---|
| **1** | *Color Masks* | Coarse | Number of identical culture colors |
| **2** | *Color Histograms* | Medium | Distribution of colors |
| **3** | *Color Average* | Medium | Similarity comparison within the same culture color |
| **4** | *Color Variance* | Fine | Similarity comparison within the same culture color |
| **5** | *Spread* | Coarse | Spatial concentration of a color |
| **6** | *Elongation* | Coarse | Shape of a color |
| **7** | *Vertical Wavelets Level 1*<br>*Horizontal Wavelets Level 1*<br>*Diagonal Wavelets Level 1* | Coarse | Vertical frequency components<br>Horizontal frequency components<br>Diagonal frequency components |
| **8** | *Vertical Wavelets Level 2*<br>*Horizontal Wavelets Level 2*<br>*Diagonal Wavelets Level 2* | Medium | Vertical frequency components<br>Horizontal frequency components<br>Diagonal frequency components |
| **9** | *Vertical Wavelets Level 3*<br>*Horizontal Wavelets Level 3*<br>*Diagonal Wavelets Level 3* | Fine | Vertical frequency components<br>Horizontal frequency components<br>Diagonal frequency components |

Table 5: Multi-resolution Image Features.

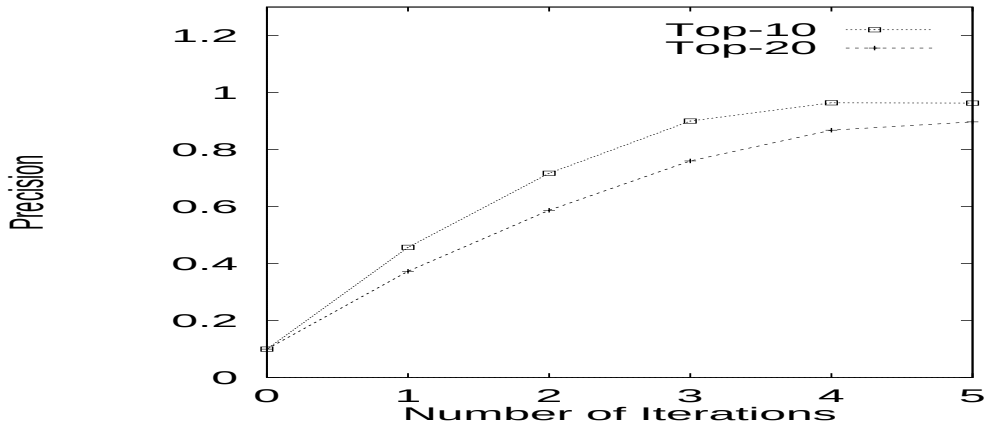



Figure 13: Average Precision of Top-10 and Top-20 Queries.

in this case because it is irrelevant.) For each of the queries, after three iterations the results were satisfactory concerning the quality of the top-10 retrieval. For top-20 retrieval, it required only one more iteration to surpass 86% precision. Finally, Figure 13 shows the average precision of the top-10 and top-20 retrieval of all queries with respect to the number of iterations.

# 5 Related Work

The existing work in query-concept learning suffers in at least one of the following three areas: *sample selection*, *feature reduction*, and *query-concept modeling*.

In most inductive learning problems studied in the AI community, samples are assumed to be taken randomly in such a way that various statistical properties can be derived conveniently. However, for interactive applications where the number of samples must be small (or impatient users might be turned away), random sampling is not suitable.

| *Categories* | Iteration 1 | | Iteration 2 | | Iteration 3 | | Iteration 4 | | Iteration 5 | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Top 10 | Top 20 | Top 10 | Top 20 | Top 10 | Top 20 | Top 10 | Top 20 | Top 10 | Top 20 |
| Architecture | 0.800 | 0.710 | 0.950 | 0.865 | 1.000 | 0.950 | 1.000 | 0.970 | 0.910 | 0.920 |
| Bears | 0.030 | 0.065 | 0.380 | 0.220 | 0.760 | 0.490 | 0.860 | 0.740 | 0.910 | 0.690 |
| Clouds | 0.260 | 0.180 | 0.420 | 0.295 | 0.780 | 0.580 | 0.910 | 0.720 | 0.980 | 0.895 |
| Flowers | 0.670 | 0.445 | 0.750 | 0.715 | 0.990 | 0.855 | 1.000 | 0.950 | 1.000 | 0.950 |
| Landscape | 0.370 | 0.260 | 0.580 | 0.430 | 0.850 | 0.575 | 0.950 | 0.795 | 0.880 | 0.900 |
| Objectionable | 0.760 | 0.670 | 0.890 | 0.815 | 1.000 | 0.900 | 0.990 | 0.955 | 0.970 | 0.950 |
| People | 0.340 | 0.250 | 0.660 | 0.550 | 0.810 | 0.635 | 1.000 | 0.815 | 0.990 | 0.840 |
| Tigers | 0.440 | 0.375 | 0.580 | 0.410 | 1.000 | 0.880 | 1.000 | 0.930 | 1.000 | 0.980 |
| Tools | 0.420 | 0.350 | 1.000 | 0.980 | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 |
| Waves | 0.480 | 0.425 | 0.960 | 0.585 | 0.810 | 0.730 | 0.930 | 0.800 | 0.990 | 0.845 |
| Average | 0.457 | 0.373 | 0.717 | 0.587 | 0.900 | 0.760 | 0.964 | 0.868 | 0.963 | 0.897 |

Table 6: Experimental Results on Image Dataset.

Relevance feedback techniques proposed by the IR (Information Retrieval) and database communities do perform non-random sampling. The study of [16] puts these query refinement approaches into three categories: *query reweighting*, *query point movement* and *query expansion*.

- *Query reweighting* and *query point movement* [7, 14, 15]. Both query reweighting and query point movement use nearest-neighbor sampling: They return top ranked objects to be marked by the user and refine the query based on the feedback. If the initial query example is good, this nearest-neighbor sampling approach works fine. However, most users may not have a good example to start a query. Refining around bad examples is analogous to trying to find oranges in the middle of an apple orchard by refining one's search to a few rows of apple trees at a time. It will take a long time to find oranges (the desired result). In addition, theoretical studies show that for the nearest neighbor approach, the number of samples needed to reach a given accuracy grows exponentially with the number of irrelevant features [10, 11], even for conjunctive concepts.
- *Query expansion* [16, 20]. The *query expansion* approach can be regarded as a multiple-instances sampling approach. The samples of the next round are selected from the neighborhood (not necessarily the nearest ones) of the positive-labeled instances of the previous round. The study of [16] shows that query expansion achieves only a slim margin of improvement (about 10% in precision/recall) over query point movement. Again, the presence of irrelevant features can make this approach perform poorly.

To reduce learning samples, *active learning* or *pool-based learning* has been introduced for choosing good samples from the unlabeled data pool. The Query by Committee (QBC) algorithm [6], uses a distribution over the hypothesis space (i.e., a distribution over all possible classifiers) and then chooses a sample to query an oracle (a user) to reduce entropy of the posterior distribution over the hypothesis space by the largest amount. QBC reduces the number of samples needed for learning a classifier, but it does not tackle the irrelevant feature problem. MEGA may be regarded as a variant of the QBC algorithm with an additional embedded[4] feature reduction step. MEGA provides an effective

[4]For query-concept learning, feature reduction must be embedded in the learning algorithm and cannot be a pre-

method for refining committee members (i.e., a $k$-CNF and a $k$-DNF hypothesis), and at the same time delimits the boundary of the sampling space for efficiently finding useful samples to further refine the committee members and the sampling boundary.

For image retrieval, the PicHunter system [3] uses Bayes' rule to predict the goal image, based upon the users' actions. The system shows that employing active learning can drastically cut down the number of iterations (up to 80% in some experiments). But, the authors also point out that their scheme is computationally intensive, since it recomputes conditional probability for all unlabeled samples after each round of user feedback and hence may not scale well with dataset size.

Finally, much traditional work suffers from model bias. Some systems (e.g., [4, 5]) assume that the overall similarity can be expressed as a weighted linear combination of similarities in features. Similarly, some systems assume that query concepts are disjunctive [20]. When a query concept does not fit the model assumption, these systems perform poorly. MEGA works well with model bias and moderately noisy feedback.

## 6 Conclusion and Relationship to Other Patent Documents

In this paper, we have presented a query-concept learning problem. We believe that this problem is important and should be attended by the database community for addressing the personalization and user-friendly aspects of information retrieval. We have proposed MEGA, a query-concept learner that learns query concepts quickly and with a small number of labeled instances. Instead of returning the "best" result after each round of relevance feedback as traditional methods do, MEGA generates intelligent samples to maximize the collection of relevant information for trimming the candidate concept-space. We have showed that the sample selection and feature reduction steps of MEGA work in a complementary manner to achieve fast convergence to query concepts.

Our experimental results showed that MEGA outperforms traditional sampling schemes in all query scenarios when the number of labeled instances is limited. In addition, we showed that MEGA can cope well with model bias and user errors. MEGA can also learn complex concepts in a high-dimensional dataset through its multi-resolution/hierarchical learning method. We believe an intelligent concept-learner like MEGA is needed for easier and faster information access. Because it is more personal and user-friendly, such a tool may open access to underprivileged users, who may not yet enjoy the full benefit of the information revolution.

To make MEGA efficient, it requires a high-dimensional access method [12] to ensure that eliminating/replacing features incurs minimum additional search overhead (Patent Document #4). MEGA can speed up its sampling step by using the support vectors generated by SVMs.

processing step, since a concept-learner may not know what a query concept is beforehand.

## References


[1] E. Chang and T. Cheng. Perception-based image retrieval. *ACM Sigmod (Demo)*, May 2001.

[2] E. Chang, B. Li, and C. Li. Towards perception-based image retrieval. *IEEE Content-Based Access of Image and Video Libraries*, pages 101–105, June 2000.

[3] I. J. Cox, M. L. Miller, T. P. Minka, T. V. Papathomas, and P. N. Yianilos. The bayesian image retrieval system, pichunter: Theory, implementation and psychological experiments. *IEEE Transaction on Image Processing (to appear)*, 2000.

[4] R. Fagin. Fuzzy queries in multimedia database systems. *ACM Sigacr-Sigmod-Sigart Symposium on Principles of Database Systems*, 1998.

[5] R. Fagin and E. L. Wimmers. A formula for incorporating weights into scoring rules. *International Conference on Database Theory*, pages 247–261, 1997.

[6] Y. Freund, H. S. Seung, E. Shamir, and N. Tishby. Selective sampling using the query by committee algorithm. *Machine Learning*, 28:133–168, 1997.

[7] Y. Ishikawa, R. Subramanya, and C. Faloutsos. Mindreader: Querying databases through multiple examples. *VLDB*, 1998.

[8] M. Kearns, M. Li, and L. Valiant. Learning boolean formulae. *Journal of ACM*, 41(6):1298–1328, 1994.

[9] M. Kearns and U. Vazirani. *An Introduction to Computational Learning Theory*. MIT Press, 1994.

[10] P. Langley and W. Iba. Average-case analysis of a nearest neighbor algorithm. *Proceedings of the* $13^{th}$ *International Joint Conference on Artificial Intelligence*, (82):889–894, 1993.

[11] P. Langley and S. Sage. Scaling to domains with many irrelevant features. *Computational Learning Theory and Narural Learning Systems*, 4, 1997.

[12] C. Li, E. Chang, H. Garcia-Molina, and G. Wiederhold. Clustering for approximate similarity queries in high-dimensional spaces. *IEEE Transaction on Knowledge and Data Engineering (to appear)*, 2001.

[13] T. Michell. *Machine Learning*. McGraw Hill, 1997.

[14] M. Ortega, Y. Rui, K. Chakrabarti, A. Warshavsky, S. Mehrotra, and T. S. Huang. Supporting ranked boolean similarity queries in mars. *IEEE Transaction on Knowledge and Data Engineering*, 10(6):905–925, December 1999.

[15] K. Porkaew, K. Chakrabarti, and S. Mehrotra. Query refinement for multimedia similarity retrieval in mars. *Proceedings of ACM Multimedia*, November 1999.

[16] K. Porkaew, S. Mehrota, and M. Ortega. Query reformulation for content based multimedia retrieval in mars. *ICMCS*, pages 747–751, 1999.

[17] Y. Rui, T. S. Huang, and S.-F. Chang. Image retrieval: Current techniques, promising directions and open issues. *Journal of Visual Communication and Image Representation*, March 1999.

[18] Y. Rui, T. S. Huang, M. Ortega, and S. Mehrotra. Relevance feedback: A power tool in interactive content-based image retrieval. *IEEE Tran on Circuits and Systems for Video Technology*, 8(5), Sept 1998.

[19] L. Valiant. A theory of learnable. *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, pages 436–445, 1984.

[20] L. Wu, C. Faloutsos, K. Sycara, and T. R. Payne. Falcon: Feedback adaptive loop for content-based retrieval. *The* $26^{th}$ *VLDB Conference*, September 2000.

[21] L. A. Zadeh. Fuzzy sets. *Information and Control*, pages 338–353, 1965.